Artificial Intelligence

Assignment 2

Solving the 8-Puzzle Problem using the A* Search Algorithm

---

1. Brief Introduction to the Problem

The 8-Puzzle consists of a 3×3 board with eight numbered tiles (1–8) and one blank cell. Tiles adjacent to the blank can slide into it, and the task is to reach a specified goal arrangement from a given (scrambled) initial arrangement using the fewest possible slides. Each board configuration is a "state", and each legal slide is an "operator" connecting one state to another, so the problem is naturally solved through state-space search.

Because a blind (uninformed) search such as BFS or DFS explores states without any sense of "direction" toward the goal, it can be very slow for this problem. An informed search algorithm that uses a heuristic to estimate distance-to-goal is far more efficient — this assignment uses the A* algorithm.

2. AI Algorithm(s) Suitable for Solving the Problem

A* Search is the most suitable algorithm for the 8-Puzzle. It is a best-first search that combines:

- g(n) — the exact cost (number of moves) taken to reach the current node n from the start node.
- h(n) — a heuristic estimate of the cost from node n to the goal. The most common heuristic for the 8-Puzzle is the sum of Manhattan distances of all tiles from their current position to their goal position.

A* evaluates every node using f(n) = g(n) + h(n) and always expands the node with the lowest f(n) value first. Since the Manhattan-distance heuristic never overestimates the true cost (it is admissible), A* is guaranteed to find the optimal (shortest) solution path.

(Other applicable algorithms include BFS — guaranteed optimal but memory-heavy and slow; and IDA — an iterative-deepening variant of A* that uses less memory.)*

3. Step-by-Step Working of the Algorithm

1. Represent the initial board as the start node. Compute h(start) using the Manhattan-distance heuristic, and set g(start) = 0, so f(start) = h(start).
2. Place the start node in the OPEN list (priority queue ordered by f). The CLOSED list (visited nodes) starts empty.
3. Repeat while OPEN is not empty: remove the node n with the smallest f(n) from OPEN.
4. If n is the goal state, stop and trace back through parent pointers to return the solution path.
5. Otherwise, move n to CLOSED and generate its successors by sliding the blank Up, Down, Left, or Right (whichever are within the board).
6. For each successor: compute g = g(n) + 1 and h using the Manhattan-distance heuristic, so f = g + h. If an equal state already exists in OPEN/CLOSED with a lower or equal f, discard the new one; otherwise add/update it in OPEN with a pointer back to n.
7. If OPEN becomes empty before the goal is found, report failure (no solution exists for that instance).

4. Flowchart / Pseudocode

4.1 Pseudocode

```
function A_STAR(start, goal):
    g[start] = 0
    f[start] = g[start] + HEURISTIC(start, goal)
    OPEN = priority_queue();  OPEN.push(start, f[start])
    CLOSED = empty_set()
    while OPEN is not empty:
        n = OPEN.pop_lowest_f()
        if n == goal:
            return RECONSTRUCT_PATH(n)
        CLOSED.add(n)
        for each successor s of n (Up, Down, Left, Right moves):
            if s in CLOSED: continue
            tentative_g = g[n] + 1
            if s not in OPEN or tentative_g < g[s]:
                g[s] = tentative_g
                f[s] = g[s] + HEURISTIC(s, goal)
                parent[s] = n
                OPEN.push_or_update(s, f[s])
    return FAILURE
```

4.2 Flowchart

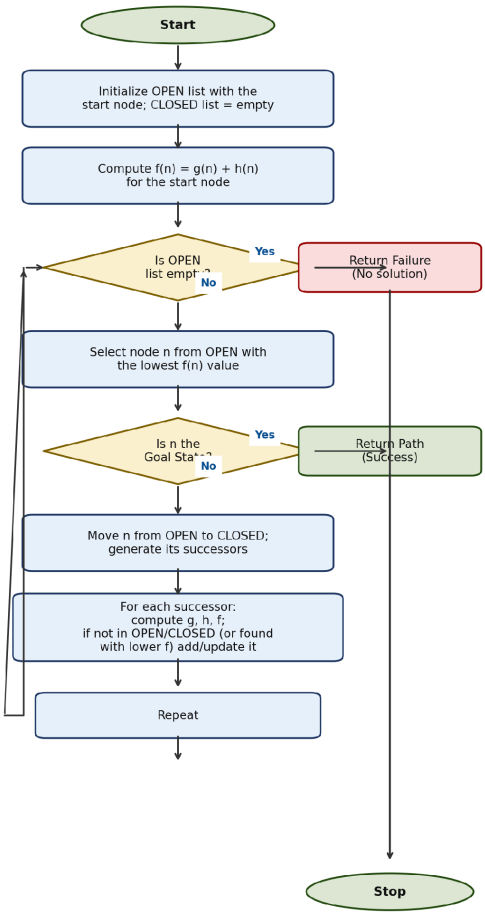

Figure: Flowchart of the A Search Algorithm*

5. Solved Example (Manual Trace)

Consider the following initial and goal states (0 = blank):

`Initial State:  1 2 3 / 4 6 0 / 7 5 8  Goal State:  1 2 3 / 4 5 6 / 7 8 0`

Heuristic used: h(n) = sum of Manhattan distances of all numbered tiles from their goal position. At the start, tiles 6, 5, and 8 are each one move away from their goal cells, giving h = 3, while tiles 1, 2, 3, 4, 7 are already correctly placed.

A* expands nodes in the following order (only the best node at each level is shown, since it is the one A* selects):

Step	Action (Blank moves)	Resulting State (row-major)	g	h	f=g+h
0	Start	1 2 3 / 4 6 0 / 7 5 8	0	3	3
1	Left (swap blank & 6)	1 2 3 / 4 0 6 / 7 5 8	1	2	3
2	Down (swap blank & 5)	1 2 3 / 4 5 6 / 7 0 8	2	1	3
3	Right (swap blank & 8)	1 2 3 / 4 5 6 / 7 8 0 — GOAL	3	0	3

At Step 3, h = 0 because every tile matches the goal arrangement, so this node is recognised as the goal. Tracing parent pointers back to the start gives the optimal solution path:

Solution: Left → Down → Right (3 moves, which is optimal since h(start) = 3 and each move reduces f by at most a bounded amount consistent with the true cost).

6. Time Complexity

In the worst case, A* has the same time complexity as an uninformed search: O(b^d), where b is the branching factor (up to 4 for the 8-Puzzle) and d is the depth of the optimal solution. However, with an informative admissible heuristic such as Manhattan distance, A* explores far fewer nodes in practice than BFS/DFS because it always expands the most promising node first — its effective branching factor is much smaller than the worst-case bound.

7. Space Complexity

A* must keep every generated node in memory (in the OPEN or CLOSED list) so it can compare f-values and reconstruct the path, giving a worst-case space complexity of O(b^d) as well — the same order as its time complexity. This is generally considered A*'s biggest drawback, and is the main motivation for memory-bounded variants such as IDA* and SMA*.

8. Advantages

- Complete: A* is guaranteed to find a solution if one exists (for a finite state space such as the 8-Puzzle).
- Optimal: with an admissible (never-overestimating) heuristic, A* always finds the least-cost / shortest solution path.
- Efficient guidance: the heuristic focuses the search toward the goal, exploring far fewer states than blind search strategies like BFS or DFS.
- Flexible: the same algorithm works for many other path-finding and planning problems simply by changing the heuristic function.

9. Limitations

- High memory usage: storing all generated nodes in OPEN/CLOSED lists can exhaust memory for large or deep state spaces (e.g., the 15-puzzle).
- Heuristic-dependent: performance depends heavily on the quality of h(n); a poor or non-admissible heuristic can make A* slow, or even non-optimal.
- Not ideal for very large search spaces: despite being better than blind search, A* can still become computationally expensive as the state space grows.
- Designing a good heuristic can itself be a non-trivial problem for more complex real-world tasks.